SIMATS ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ASSESSMENT TOOL 1 — EXPERIMENTS REPORT

Course Name & Code:	**Cryptography and Network Security (CSA51)**
Course Outcome Covered:	CO3: Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks (BL4, BL5)
Student Name:	**T.Lakshmi Prasad Naidu**
Register Number:	**192324187**

Code of Conduct Certification:

I, **T.Lakshmi Prasad Naidu (Reg No: 192324187)***, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____

Experiment 1: Password Hashing with Salting and Key Stretching

Objective: Design and implement a program in Python to store and verify passwords using salted hashing (PBKDF2). Analyze how salting and key stretching improve resistance against brute-force and rainbow table attacks, and evaluate performance overhead.

Python Implementation

```
import hashlib
import os
import time

def hash_password(password: str, iterations: int = 100000) -> tuple[bytes, bytes]:
    """Generates a cryptographically secure salt and hashes the password using
PBKDF2."""
    salt = os.urandom(16)  # 128-bit cryptographically secure salt
    hashed_key = hashlib.pbkdf2_hmac(
        'sha256',
        password.encode('utf-8'),
        salt,
        iterations
    )
    return salt, hashed_key

def verify_password(stored_salt: bytes, stored_key: bytes, password_attempt: str,
```

```
iterations: int = 100000) -> bool:
    """Verifies a candidate password against the stored salt and key."""
    new_key = hashlib.pbkdf2_hmac(
        'sha256',
        password_attempt.encode('utf-8'),
        stored_salt,
        iterations
    )
    return hashlib.compare_digest(stored_key, new_key)

# Demonstration & Benchmark
if __name__ == "__main__":
    password = "SecurePassword123!"

    print("=== PBKDF2 Performance Benchmark ===")
    for iterations in [1_000, 100_000, 500_000]:
        start = time.perf_counter()
        salt, key = hash_password(password, iterations)
        elapsed = time.perf_counter() - start

        is_valid = verify_password(salt, key, password, iterations)
        print(f"Iterations: {iterations:<7} | Time: {elapsed:.4f}s | Verified:
{is_valid}")
```

Analysis of Results

- **Salting Impact:** A unique 128-bit random salt guarantees that even identical user passwords produce completely distinct hash outputs. This completely invalidates precomputed lookup tables (Rainbow Tables), forcing an attacker to generate dedicated lookup tables for every individual target user salt.
- **Key Stretching (PBKDF2 Iterations):** By tuning the iteration parameter (e.g., 100,000+ loops), the function deliberately slows down calculation times. While negligible for legitimate single logins, this exponentially inflates computation times for offline GPU/ASIC brute-force dictionary attacks.
- **Performance Overhead Evaluation:** Increasing iterations introduces operational CPU latency on authentication servers. Balancing iteration depth (e.g., 100,000 to 600,000 iterations for PBKDF2-HMAC-SHA256) provides strong resistance against brute-force attacks while preserving smooth user authentication experience.

Experiment 2: Implementation of HMAC for Message Integrity

Objective: Develop a Python application to implement HMAC using SHA-256 for message authentication. Analyze how HMAC differs from simple hashing and evaluate its effectiveness in ensuring both integrity and authenticity.

Python Implementation

```
import hmac
```

```
import hashlib

def generate_hmac(secret_key: bytes, message: str) -> str:
    """Generates an HMAC digest for a message using SHA-256."""
    return hmac.new(secret_key, message.encode('utf-8'), hashlib.sha256).hexdigest()

def verify_hmac(secret_key: bytes, message: str, received_mac: str) -> bool:
    """Verifies HMAC using constant-time comparison to prevent timing attacks."""
    expected_mac = generate_hmac(secret_key, message)
    return hmac.compare_digest(expected_mac, received_mac)

# Demonstration
if __name__ == "__main__":
    shared_key = b"super_secret_shared_key_123"
    original_message = "Transfer $1000 to Account #98765"

    # 1. Sender generates HMAC
    mac = generate_hmac(shared_key, original_message)
    print(f"Original Message: '{original_message}'")
    print(f"Generated HMAC:   {mac}")

    # 2. Receiver validates legitimate untampered message
    is_valid = verify_hmac(shared_key, original_message, mac)
    print(f"Verification Result (Untampered): {is_valid}")

    # 3. Receiver detects tampered message
    tampered_message = "Transfer $9000 to Account #98765"
    is_tampered_valid = verify_hmac(shared_key, tampered_message, mac)
    print(f"Verification Result (Tampered):   {is_tampered_valid}")
```

Analysis of Results

- **Simple Hashing vs. HMAC:** A simple hash H(M) ensures message integrity against accidental transmission errors, but cannot protect against active attackers who can modify M to M' and compute a new H(M'). HMAC incorporates a shared secret key K with nested hashing, proving that the message was generated by a holder of K (Authenticity) and has not been altered (Integrity).
- **Length Extension Attack Immunity:** Simple constructions like H(K || M) are vulnerable to length extension attacks inherent in Merkle-Damgård hash functions. HMAC utilizes two rounds of hashing with inner and outer padding constants (ipad and opad), structurally neutralizing length extension vulnerabilities.

Experiment 3: Simulation of Certificate Chain Validation

Objective: Create a Python program to simulate X.509 certificate chain verification. Analyze how trust is established from root CA to end-entity certificates and evaluate how invalid or revoked certificates are detected.

Python Implementation

```
import datetime
import hashlib

class MockCertificate:
    def __init__(self, subject: str, issuer: str, public_key: str, signing_key: str,
valid_days: int = 30):
        self.subject = subject
        self.issuer = issuer
        self.public_key = public_key
        self.valid_from = datetime.datetime.now(datetime.timezone.utc)
        self.valid_to = self.valid_from + datetime.timedelta(days=valid_days)
        self.signature = self._sign(signing_key)

    def _sign(self, signing_key: str) -> str:
        data = f"{self.subject}|{self.issuer}|{self.public_key}|{self.valid_to}"
        return hashlib.sha256((data + signing_key).encode()).hexdigest()

    def is_expired(self) -> bool:
        return datetime.datetime.now(datetime.timezone.utc) > self.valid_to

def verify_certificate_chain(chain: list[MockCertificate], crl: list[str]) ->
tuple[bool, str]:
    """Validates an X.509 chain from End-Entity to Root CA."""
    for i in range(len(chain)):
        cert = chain[i]

        # 1. Check Revocation
        if cert.subject in crl:
            return False, f"Certificate '{cert.subject}' is REVOKED."

        # 2. Check Expiration
        if cert.is_expired():
            return False, f"Certificate '{cert.subject}' is EXPIRED."

        # 3. Verify Signature against Parent Key (or Self if Root)
        parent_cert = chain[i+1] if i + 1 < len(chain) else cert
        expected_sig = hashlib.sha256(
f"{cert.subject}|{cert.issuer}|{cert.public_key}|{cert.valid_to}{parent_cert.public_key
}".encode()
        ).hexdigest()

        if cert.signature != expected_sig:
            return False, f"Invalid signature on certificate '{cert.subject}'."

    return True, "Certificate chain successfully validated!"

# Simulation Execution
if __name__ == "__main__":
```

```
    root_key, inter_key, leaf_key = "root_priv_key", "inter_priv_key", "leaf_priv_key"

    # Construct Hierarchical Certificates
    root_cert = MockCertificate("Root CA", "Root CA", "root_pub_key", root_key,
valid_days=365)
    inter_cert = MockCertificate("Intermediate CA", "Root CA", "inter_pub_key",
root_key, valid_days=180)
    leaf_cert = MockCertificate("example.com", "Intermediate CA", "leaf_pub_key",
inter_key, valid_days=30)

    chain = [leaf_cert, inter_cert, root_cert]
    crl_list = ["malicious_site.com"]

    status, message = verify_certificate_chain(chain, crl_list)
    print(f"Validation Status: {status} | Details: {message}")
```

Analysis of Results

- **Hierarchical Trust Model:** Trust is anchored in a highly secured Root CA. The Root CA delegates issuing authority to Intermediate CAs, which in turn issue Leaf / End-Entity certificates. This isolates the Root CA offline, protecting system security.
- **Verification Steps:** To establish trust, client software must recursively verify: (1) Valid cryptographic digital signatures up to a trusted Root CA, (2) Temporal validity bounds (notBefore / notAfter), and (3) Active revocation status via CRLs or OCSP endpoints.

Experiment 4: Replay Attack Simulation and Prevention

Objective: Develop a Python program to simulate a replay attack in an authentication system. Implement a prevention mechanism using nonces or timestamps. Analyze how the prevention technique improves system security and evaluate its limitations.

Python Implementation

```
import time
import secrets
import hashlib

class AuthenticationServer:
    def __init__(self, time_window_seconds: int = 5):
        self.used_nonces = set()
        self.time_window = time_window_seconds
        self.secret_key = "server_shared_secret"

    def authenticate(self, username: str, timestamp: float, nonce: str, token: str) ->
tuple[bool, str]:
        current_time = time.time()

        # 1. Check timestamp freshness window
```

```
        if abs(current_time - timestamp) > self.time_window:
            return False, "REJECTED: Timestamp expired/out of valid window."

        # 2. Check for Nonce reuse
        if nonce in self.used_nonces:
            return False, "REJECTED: Replay attack detected (Nonce already used)."

        # 3. Verify token signature
        expected_token = hashlib.sha256(
            f"{username}{timestamp}{nonce}{self.secret_key}".encode()
        ).hexdigest()

        if not hmac.compare_digest(expected_token, token):
            return False, "REJECTED: Invalid authentication token."

        # Store used nonce
        self.used_nonces.add(nonce)
        return True, "SUCCESS: Authenticated successfully."

# Simulation Execution
if __name__ == "__main__":
    server = AuthenticationServer(time_window_seconds=3)
    username = "alice"
    secret = "server_shared_secret"

    # 1. Legitimate Client Request Creation
    ts = time.time()
    nonce = secrets.token_hex(8)
    token = hashlib.sha256(f"{username}{ts}{nonce}{secret}".encode()).hexdigest()

    # 2. First Transmission (Success)
    status1, msg1 = server.authenticate(username, ts, nonce, token)
    print(f"Attempt 1 (Legitimate): {msg1}")

    # 3. Attacker Intercepts and Replays Exact Payload (Blocked)
    status2, msg2 = server.authenticate(username, ts, nonce, token)
    print(f"Attempt 2 (Replayed):   {msg2}")
```

Analysis of Results

- **Prevention Mechanics:** Nonces ensure every payload is uniquely identifiable. Once consumed, the server rejects duplicate nonces. Combining nonces with short timestamp validity windows prevents unlimited accumulation of cached nonces on the server.
- **System Limitations:** Timestamp checking requires time synchronization (NTP) between client and server; clock skew can cause legitimate requests to be rejected. Additionally, storing nonces during high-throughput DDoS attacks can increase server memory utilization.

Experiment 5: Performance Comparison of Digital Signature Algorithms

Objective: Implement and compare two digital signature algorithms (RSA and ECDSA) in Python. Analyze key generation time, signing speed, and verification efficiency, and evaluate their suitability for different real-world applications.

Python Implementation

```
import time
from cryptography.hazmat.primitives.asymmetric import rsa, ec
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import padding

def benchmark_rsa():
    # 1. Key Generation
    t0 = time.perf_counter()
    private_key = rsa.generate_private_key(public_exponent=65537, key_size=2048)
    t_keygen = time.perf_counter() - t0

    public_key = private_key.public_key()
    message = b"Authenticated transaction payload"

    # 2. Signing
    t0 = time.perf_counter()
    signature = private_key.sign(message, padding.PKCS1v15(), hashes.SHA256())
    t_sign = time.perf_counter() - t0

    # 3. Verification
    t0 = time.perf_counter()
    public_key.verify(signature, message, padding.PKCS1v15(), hashes.SHA256())
    t_verify = time.perf_counter() - t0

    return t_keygen, t_sign, t_verify, len(signature)

def benchmark_ecdsa():
    # 1. Key Generation
    t0 = time.perf_counter()
    private_key = ec.generate_private_key(ec.SECP256R1())
    t_keygen = time.perf_counter() - t0

    public_key = private_key.public_key()
    message = b"Authenticated transaction payload"

    # 2. Signing
    t0 = time.perf_counter()
    signature = private_key.sign(message, ec.ECDSA(hashes.SHA256()))
    t_sign = time.perf_counter() - t0
```

```
    # 3. Verification
    t0 = time.perf_counter()
    public_key.verify(signature, message, ec.ECDSA(hashes.SHA256()))
    t_verify = time.perf_counter() - t0

    return t_keygen, t_sign, t_verify, len(signature)

# Benchmark Execution
if __name__ == "__main__":
    rsa_kg, rsa_s, rsa_v, rsa_sz = benchmark_rsa()
    ec_kg, ec_s, ec_v, ec_sz = benchmark_ecdsa()

    print(f"{'Metric':<22} | {'RSA-2048':<15} | {'ECDSA (P-256)':<15}")
    print("-" * 58)
    print(f"{'KeyGen Time (s)':<22} | {rsa_kg:<15.5f} | {ec_kg:<15.5f}")
    print(f"{'Signing Time (s)':<22} | {rsa_s:<15.5f} | {ec_s:<15.5f}")
    print(f"{'Verify Time (s)':<22} | {rsa_v:<15.5f} | {ec_v:<15.5f}")
    print(f"{'Signature Size (bytes)':<22} | {rsa_sz:<15} | {ec_sz:<15}")
```

Comparative Evaluation Table

Metric / Feature	RSA (2048-bit)	ECDSA (SECP256R1)
Key Generation	Slow (requires generating large prime pairs)	Extremely Fast (elliptic curve point scaling)
Signing Speed	Slower (modular exponentiation with private exponent d)	Fast (lightweight scalar multiplication)
Verification Speed	Extremely Fast (small public exponent e=65537)	Moderately Fast
Signature & Key Size	Large (256-byte signature, 2048-bit key)	Compact (~64-72 byte signature, 256-bit key)

Real-World Suitability

- **RSA (2048-bit):** Optimal for scenarios requiring extremely fast signature verification where signature length is not critical, such as firmware code signing, static document verification, and OS boot loader checks.
- **ECDSA (P-256):** Ideal for bandwidth-constrained, resource-limited devices, mobile networks, IoT, TLS 1.3 handshakes, and blockchain transactions due to its smaller signature size and efficient key generation.